

DOCUMENTO DE ARQUITECTURA DE SOFTWARE

Sistema Bancario Micro-Políglota Resiliente

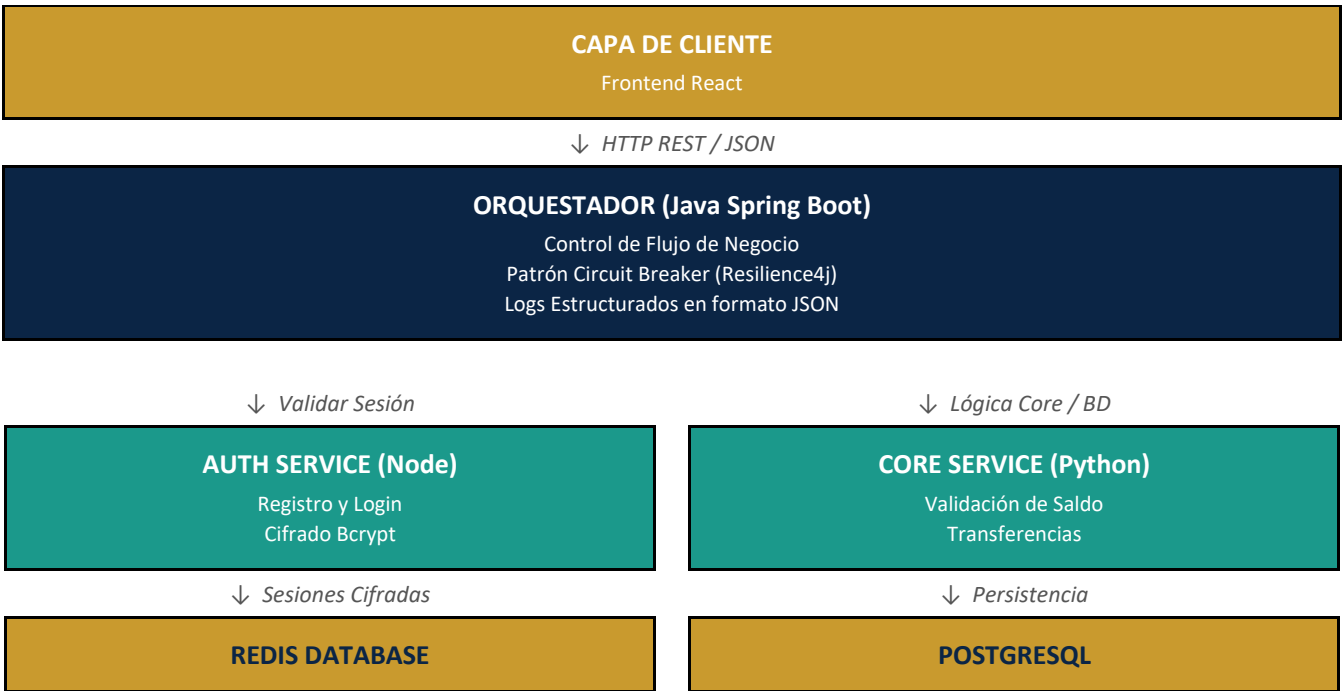
Formato: Versión 1.0

1. Introducción y Objetivos

El presente documento define la arquitectura y el diseño técnico del ecosistema de microservicios solicitado en la prueba técnica backend. El objetivo principal es construir una plataforma bancaria altamente escalable, desacoplada y capaz de manejar operaciones críticas de manera segura y resiliente bajo un esquema tecnológico políglota.

2. Vista General de la Arquitectura (Modelo C4 - Contenedores)

Se optó por una arquitectura de microservicios distribuidos coordinados mediante un patrón de Orquestación Centralizada. Esto asegura que las transacciones complejas sean gobernadas por un único punto de control, aislando las responsabilidades específicas de cada lenguaje.



3. Desglose de Componentes Tecnológicos

Componente	Tecnología	Responsabilidad Principal
Auth Service	Node.js / Express	Autenticación de usuarios, registro, login y logout. Cifrado con Bcrypt y manejo de sesiones en Redis.
Orchestrator	Java Spring Boot	Puerta de entrada principal, coordinación transaccional, Circuit Breaker con Resilience4j y logs estructurados.
Core Service	Python FastAPI	Validación de negocio profunda, persistencia física de cuentas y movimientos en la base de datos PostgreSQL.

4. Diagrama de Secuencia: Flujo de Transferencia Línea Base

El siguiente flujo detalla la secuencia de operaciones y validaciones cruzadas necesarias para ejecutar una transferencia por número telefónico con total seguridad:

1	Usuario/Front	POST /transfers → Orquestador (Monto, Destino)
2	Orquestador	Valida Token y Sesión con Auth Service (Node)
3	Auth (Node)	Responde: Sesión OK
4	Orquestador	Ejecuta Transferencia hacia Core Service (Monto, Destino)
5	Core (Python)	Consulta Saldo en Base de Datos
6	Base de Datos	Responde: Saldo OK
7	Core (Python)	Actualiza Cuentas en Base de Datos
8	Base de Datos	Responde: Confirmación
9	Core (Python)	Responde al Orquestador: HTTP 201 (Creado)
10	Orquestador	Responde al Usuario/Front: Éxito (JSON)

5. Resiliencia, Seguridad y Buenas Prácticas

5.1. Manejo de Fallas con Circuit Breaker

El orquestador en Java Spring Boot implementa un Circuit Breaker configurado estratégicamente sobre las llamadas salientes a los servicios de Auth y Core. Si el microservicio de Python experimenta degradación o caídas accidentales, el circuito cambia al estado OPEN tras un umbral del 50% de fallas en una ventana de tiempo, redirigiendo la petición hacia un método de Fallback controlado para evitar cuellos de botella y huelgas de hilos de ejecución en el servidor web.

5.2. Logs Estructurados

Cada petición que entra al sistema es marcada en el Orquestador con un identificador único (Correlation-ID). Este identificador es inyectado en las cabeceras HTTP hacia los servicios secundarios. Los logs se emiten en formato estructurado JSON:

```
{ "timestamp": "2026-05-16T11:05:00Z", "level": "INFO", "correlation_id": "tx-89234-usr9", "service": "orchestrator-java", "message": "Iniciando transferencia hacia número: 3001234567", "amount": 25000.00 }
```

5.3. Persistencia Relacional vs No Relacional

Redis (No-SQL)

- Utilizado con exclusividad para almacenar tokens de sesión activos y listas negras temporales de tokens revocados (logout).
- Justificación: bajísima latencia de lectura/escritura en memoria RAM.

PostgreSQL (Relacional)

- Resguarda la verdad transaccional de los usuarios, cuentas y el histórico inmutable de movimientos.
- Garantiza consistencia ACID estricta para transacciones monetarias financieras.